

第 5 讲：蒙特卡洛方法

第 5 讲：蒙特卡洛方法

在第 4 讲中，我们介绍了基于系统模型求解最优策略的算法（值迭代与策略迭代）。从本章开始，我们将介绍不依赖系统模型的**无模型（model-free）**强化学习算法。

无模型强化学习的基本思想是：如果没有模型，就必须有数据；如果没有数据，就必须有模型。若两者皆无，则无法找到最优策略。强化学习中的“数据”通常指智能体与环境的交互经验。

本章基于**蒙特卡洛（Monte Carlo, MC）**方法，从经验样本中学习最优策略。我们首先介绍均值估计问题，说明如何从数据而非模型进行学习——状态价值和动作价值都是回报的期望，估计它们本质上就是均值估计问题。随后，我们依次介绍三种 MC 算法：MC Basic、MC Exploring Starts 和 MC ϵ -Greedy。

5.1 动机示例：均值估计

我们以均值估计问题为例，展示如何从数据而非模型进行学习。考虑一个取值于有限实数集 $\mathcal{X}$ 的随机变量 X ，任务是计算 X 的期望 $\mathbb{E}[X]$ 。有两种方法可以实现。

基于模型的方法：若 X 的概率分布已知，则可以直接根据期望的定义计算：

$$\mathbb{E}[X] = \sum_{x \in \mathcal{X}} p(x)x.$$

无模型方法：若概率分布未知，但有一些样本 $\{x_1, x_2, \dots, x_n\}$ ，则期望可以近似为：

$$\mathbb{E}[X] \approx \bar{x} = \frac{1}{n} \sum_{j=1}^n x_j.$$

当 n 较小时，该近似可能不准确；但随着 n 增大，近似越来越准确。当 $n \rightarrow \infty$ 时，有 $\bar{x} \rightarrow \mathbb{E}[X]$ 。这由大数定律保证。

定理 5.1 大数定律

对于随机变量 X ，设 $\{x_i\}_{i=1}^n$ 为独立同分布（i.i.d.）样本，令 $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$ 为样本均值。则：

$$\mathbb{E}[\bar{x}] = \mathbb{E}[X], \quad \text{var}[\bar{x}] = \frac{1}{n} \text{var}[X].$$

上述两式表明 $\bar{x}$ 是 $\mathbb{E}[X]$ 的无偏估计，且其方差随 n 增大而趋于零。

证明

首先， $\mathbb{E}[\bar{x}] = \mathbb{E}[\sum_{i=1}^n x_i/n] = \sum_{i=1}^n \mathbb{E}[x_i]/n = \mathbb{E}[X]$ ，最后一步利用了样本同分布的性质（即 $\mathbb{E}[x_i] = \mathbb{E}[X]$ ）。

其次， $\text{var}(\bar{x}) = \text{var}[\sum_{i=1}^n x_i/n] = \sum_{i=1}^n \text{var}[x_i]/n^2 = (n \cdot \text{var}[X])/n^2 = \text{var}[X]/n$ ，其中第二步

利用了样本相互独立，第三步利用了样本同分布（即 $\text{var}[x_i] = \text{var}[X]$ ）。

下面通过抛硬币的例子来说明上述两种方法。设随机变量 X 表示硬币落地时朝上的一面：正面朝上时 $X = +1$ ，反面朝上时 $X = -1$ 。假设 X 的真实概率分布为：

$$p(X = 1) = 0.5, \quad p(X = -1) = 0.5.$$

若概率分布事先已知，可以直接计算均值： $\mathbb{E}[X] = 0.5 \times 1 + 0.5 \times (-1) = 0$ 。

若概率分布未知，可以多次抛硬币并记录采样结果 $\{x_i\}_{i=1}^n$ ，通过计算样本均值得到期望的估计。随着样本数量增加，估计均值越来越准确。

备注

用于均值估计的样本必须是独立同分布（i.i.d.）的。否则，若采样值之间存在相关性，可能无法正确估计期望。一个极端情形是：所有采样值都与第一个样本相同，则无论使用多少样本，样本均值始终等于第一个样本的值。

5.2 MC Basic: 最简单的 MC 强化学习算法

本节介绍第一个也是最简单的基于 MC 的强化学习算法——MC Basic。该算法通过将策略迭代（第 4.2 节）中基于模型的策略评估步骤替换为无模型的 MC 估计步骤而得到。

5.2.1 将策略迭代改造为无模型方法

策略迭代算法每次迭代包含两步：

- **策略评估**：通过求解 $v_{\pi_k} = \mathbf{r}_{\pi_k} + \gamma \mathbf{P}_{\pi_k} v_{\pi_k}$ 来计算 v_{π_k} ；
- **策略改进**：求解贪心策略 $\pi_{k+1} = \arg \max_{\pi} (\mathbf{r}_{\pi} + \gamma \mathbf{P}_{\pi} v_{\pi_k})$ 。

策略改进步骤的元素形式为：

$$\pi_{k+1}(s) = \arg \max_{\pi} \sum_a \pi(a | s) \left[\sum_r p(r | s, a) r + \gamma \sum_{s'} p(s' | s, a) v_{\pi_k}(s') \right] = \arg \max_{\pi} \sum_a \pi(a | s) q_{\pi_k}(s, a), \quad s \in \mathcal{S}.$$

动作价值在这两步中处于核心地位。计算动作价值有两种方法：

基于模型的方法（策略迭代所采用）：先求解贝尔曼方程得到 v_{π_k} ，再通过模型计算 $q_{\pi_k}(s, a)$ ：

$$q_{\pi_k}(s, a) = \sum_r p(r | s, a) r + \gamma \sum_{s'} p(s' | s, a) v_{\pi_k}(s'). \quad (1)$$

这需要已知系统模型 $\{p(r | s, a), p(s' | s, a)\}$ 。

无模型方法：动作价值的定义为

$$q_{\pi_k}(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a] = \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots | S_t = s, A_t = a],$$

即从 (s, a) 出发的期望回报。由于 $q_{\pi_k}(s, a)$ 是期望，可以用 MC 方法估计：从 (s, a) 出发，遵循策略 π_k 与环境交互，收集 n 个 episode，每个 episode 的回报为 $g_{\pi_k}^{(i)}(s, a)$ ，则：

$$q_{\pi_k}(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a] \approx \frac{1}{n} \sum_{i=1}^n g_{\pi_k}^{(i)}(s, a). \quad (2)$$

根据大数定律，若 episode 数量 n 足够大，该近似将足够准确。

MC 强化学习的核心思想就是：用无模型方法估计动作价值（式(2)），替代策略迭代中基于模型的方法（式(1)）。

5.2.2 MC Basic 算法

现在可以给出第一个 MC 强化学习算法。从初始策略 π_0 出发, 在第 k 次迭代中 ($k = 0, 1, 2, \dots$) 执行以下两步:

步骤 1: 策略评估。 对所有 (s, a) 估计 $q_{\pi_k}(s, a)$ 。具体地, 对每个 (s, a) , 收集足够多的 episode, 用这些 episode 的平均回报 $q_k(s, a)$ 来近似 $q_{\pi_k}(s, a)$ 。

步骤 2: 策略改进。 对所有 $s \in \mathcal{S}$, 求解 $\pi_{k+1}(s) = \arg \max_{\pi} \sum_a \pi(a | s) q_k(s, a)$ 。贪心最优策略为 $\pi_{k+1}(a_k^* | s) = 1$, 其中 $a_k^* = \arg \max_a q_k(s, a)$ 。

定义

算法 5.1: MC Basic (策略迭代的无模型变体)

初始化: 初始策略 π_0 。

目标: 搜索最优策略。

对于第 k 次迭代 ($k = 0, 1, 2, \dots$), 执行:

- 对于每个状态 $s \in \mathcal{S}$, 执行:
 - 对于每个动作 $a \in \mathcal{A}(s)$, 执行:
 - * 收集足够多的从 (s, a) 出发、遵循 π_k 的 episode。
 - * **策略评估:** $q_{\pi_k}(s, a) \approx q_k(s, a) =$ 所有从 (s, a) 出发的 episode 的平均回报。
 - * **策略改进:** $a_k^*(s) = \arg \max_a q_k(s, a)$,
 $\pi_{k+1}(a | s) = 1$ 若 $a = a_k^*$, 否则 $\pi_{k+1}(a | s) = 0$ 。

MC Basic 与策略迭代非常相似, 唯一区别在于: MC Basic 直接从经验样本中计算动作价值, 而策略迭代先计算状态价值, 再基于系统模型计算动作价值。需要注意的是, 无模型算法必须直接估计动作价值——若估计的是状态价值, 仍需借助系统模型 (式(1)) 才能得到动作价值。

由于策略迭代是收敛的, 当有足够样本时 MC Basic 也是收敛的。即对于每个 (s, a) , 若有足够多从 (s, a) 出发的 episode, 则这些 episode 的平均回报可以准确近似 (s, a) 的动作价值。实际中通常无法为每个 (s, a) 提供足够的 episode, 因此动作价值的近似可能不够准确; 但算法通常仍能工作——这类似于截断策略迭代的情形。

MC Basic 过于简单, 样本效率低下, 因此不实用。引入该算法的目的是让读者把握 MC 强化学习的核心思想, 为理解后续更复杂的算法打下基础。

5.2.3 示例分析

简单示例: 逐步实现

考虑图 5.3 所示的 3×3 网格世界。奖励设置为 $r_{\text{boundary}} = r_{\text{forbidden}} = -1$, $r_{\text{target}} = 1$, 折扣因子 $\gamma = 0.9$ 。初始策略 π_0 在 s_1 和 s_3 处并非最优。

下面仅展示 s_1 处的动作价值计算。 s_1 处有 5 个可能的动作。由于该示例的策略和模型都是确定性的, 多次运行会产生相同的轨迹, 因此每个动作价值的估计只需要一个 episode。

遵循 π_0 , 分别从 $(s_1, a_1), (s_1, a_2), \dots, (s_1, a_5)$ 出发, 得到以下 episode:

- 从 (s_1, a_1) 出发: episode 为 $s_1 \xrightarrow{a_1} s_1 \xrightarrow{a_1} s_1 \xrightarrow{a_1} \dots$, 动作价值为

$$q_{\pi_0}(s_1, a_1) = -1 + \gamma(-1) + \gamma^2(-1) + \dots = -\frac{1}{1-\gamma}.$$

- 从 (s_1, a_2) 出发: episode 为 $s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_3} \dots$, 动作价值为

$$q_{\pi_0}(s_1, a_2) = 0 + \gamma \cdot 0 + \gamma^2 \cdot 0 + \gamma^3 \cdot 1 + \gamma^4 \cdot 1 + \dots = \frac{\gamma^3}{1-\gamma}.$$

- 从 (s_1, a_3) 出发: episode 为 $s_1 \xrightarrow{a_3} s_4 \xrightarrow{a_2} s_5 \xrightarrow{a_3} \dots$, 动作价值为

$$q_{\pi_0}(s_1, a_3) = 0 + \gamma \cdot 0 + \gamma^2 \cdot 0 + \gamma^3 \cdot 1 + \gamma^4 \cdot 1 + \dots = \frac{\gamma^3}{1 - \gamma}.$$

- 从 (s_1, a_4) 出发: episode 为 $s_1 \xrightarrow{a_4} s_1 \xrightarrow{a_1} s_1 \xrightarrow{a_1} \dots$, 动作价值为

$$q_{\pi_0}(s_1, a_4) = -1 + \gamma(-1) + \gamma^2(-1) + \dots = -\frac{1}{1 - \gamma}.$$

- 从 (s_1, a_5) 出发: episode 为 $s_1 \xrightarrow{a_5} s_1 \xrightarrow{a_1} s_1 \xrightarrow{a_1} \dots$, 动作价值为

$$q_{\pi_0}(s_1, a_5) = 0 + \gamma(-1) + \gamma^2(-1) + \dots = -\frac{\gamma}{1 - \gamma}.$$

比较五个动作价值可知, $q_{\pi_0}(s_1, a_2) = q_{\pi_0}(s_1, a_3) = \frac{\gamma^3}{1 - \gamma} > 0$ 是最大值。因此改进后的策略为 $\pi_1(a_2 | s_1) = 1$ 或 $\pi_1(a_3 | s_1) = 1$ 。显然该改进策略在 s_1 处是最优的。该简单示例仅需一次迭代即可获得最优策略。

综合示例: Episode 长度与稀疏奖励

接下来通过 5×5 网格世界(图 5.4)讨论 MC Basic 的一些有趣性质。奖励设置为 $r_{\text{boundary}} = -1$, $r_{\text{forbidden}} = -10$, $r_{\text{target}} = 1$, $\gamma = 0.9$ 。

首先, episode 长度对最终最优策略有重要影响。当 episode 长度太短时, 策略和价值估计都不是最优的:

- 当 episode 长度为 1 时, 只有与目标相邻的状态具有非零价值, 其他状态价值为零——因为每个 episode 太短, 无法到达目标或获得正奖励。
- 随着 episode 长度增加, 策略和价值估计逐渐逼近最优值。当 episode 长度达到 15 时, 可以获得最优策略; 当长度达到 30 时, 价值估计也已非常接近最优; 当长度为 100 时, 价值估计几乎完全准确。

随着 episode 长度增加, 一个有趣的模式出现: 离目标更近的状态比离目标更远的状态更早获得非零价值。原因是: 从某状态出发, 智能体至少需要走一定步数才能到达目标并获得正奖励。若 episode 长度小于所需的最小步数, 回报必然为零, 估计的状态价值也为零。在该示例中, episode 长度必须不小于 15 (从最左下角状态到达目标所需的最小步数)。

备注

虽然上述分析表明每个 episode 必须足够长, 但 episode 并不需要无限长。当 episode 长度为 30 时, 算法已能找到最优策略, 尽管价值估计尚未完全达到最优值。

上述分析还与一个重要奖励设计问题——**稀疏奖励 (sparse reward)**——密切相关。稀疏奖励是指在到达目标之前无法获得任何正奖励的情形。稀疏奖励设置要求 episode 足够长才能到达目标, 当状态空间很大时这一要求难以满足, 从而降低学习效率。解决该问题的一种简单技术是设计**非稀疏奖励**: 例如在上述网格世界中, 可以重新设计奖励, 使智能体到达目标附近状态时就能获得小的正奖励, 从而在目标周围形成“吸引场”, 帮助智能体更容易地找到目标。

5.3 MC Exploring Starts

接下来扩展 MC Basic 算法, 得到另一个稍微复杂但样本效率更高的 MC 强化学习算法。

5.3.1 更高效地利用样本

MC 强化学习的一个重要方面是如何更高效地利用样本。假设有遵循策略 π 的一个 episode:

$$s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_4} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots$$

其中下标表示状态或动作的索引而非时间步。每个状态-动作对在 episode 中出现一次称为对该状态-动作对的一次访问 (**visit**)。

采用不同的策略来利用这些访问：

初始访问策略 (initial-visit)：只使用 episode 来估计其起始状态-动作对的动作价值。上例中仅估计 (s_1, a_2) 的价值。MC Basic 采用的就是这种策略。但该策略样本效率不高——episode 中还访问了 $(s_2, a_4), (s_2, a_3), (s_5, a_1)$ 等状态-动作对，这些访问也可以用来估计相应的动作价值。

具体地，可以将原始 episode 分解为多个子 episode：

$$\begin{aligned} s_1 &\xrightarrow{a_2} s_2 \xrightarrow{a_4} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \cdots \quad [\text{原始 episode}] \\ s_2 &\xrightarrow{a_4} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \cdots \quad [\text{从 } (s_2, a_4) \text{ 出发的子 episode}] \\ s_1 &\xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \cdots \quad [\text{从 } (s_1, a_2) \text{ 出发的子 episode}] \\ s_2 &\xrightarrow{a_3} s_5 \xrightarrow{a_1} \cdots \quad [\text{从 } (s_2, a_3) \text{ 出发的子 episode}] \\ s_5 &\xrightarrow{a_1} \cdots \quad [\text{从 } (s_5, a_1) \text{ 出发的子 episode}] \end{aligned}$$

状态-动作对被访问后生成的轨迹可以视为新的 episode，用于估计更多的动作价值，从而更高效地利用样本。

此外，一个状态-动作对在一个 episode 中可能被多次访问。例如上面 (s_1, a_2) 被访问了两次。若只计算首次访问，称为**首次访问策略 (first-visit)**；若计算每次访问，称为**每次访问策略 (every-visit)**。

就样本利用效率而言，每次访问策略最优。若一个 episode 足够长以至于能多次访问所有状态-动作对，则仅凭这个 episode 就足以估计所有动作价值。但每次访问策略获得的样本之间存在相关性——第二次访问开始的轨迹仅仅是第一次访问开始的轨迹的子集。不过若两次访问在轨迹中相距足够远，相关性不会太强。

补充内容：多次访问相关性的量化分析

下面从数学上量化每次访问策略中多次访问之间的相关性，并给出首次访问与每次访问的方差比较。

1. 同 episode 内两次访问的回报分解

设在同一 episode 中，状态-动作对 (s, a) 在第 i 步和第 j 步被访问 ($i < j$)，两次访问间隔 $d = j - i$ 步。对应的回报分别为 G_i 和 G_j 。由回报的递归定义， G_i 可分解为从第 i 步到第 j 步的 d 步累积回报加上第 j 步之后的剩余部分：

$$G_i = \underbrace{\sum_{k=0}^{d-1} \gamma^k R_{i+k+1}}_{A_{ij}} + \gamma^d G_j, \quad (3)$$

其中 A_{ij} 为 (s, a) 在第 i 步访问后到第 j 步访问前累积的 d 步折扣回报。注意 A_{ij} 不包含 s_j 处的奖励（该奖励已纳入 G_j ），因为 G_j 的定义时刻从 $t = j$ 开始，即 $G_j = R_{j+1} + \gamma R_{j+2} + \cdots$ 。

2. 相关性的定量表达

由式(3)，可计算 G_i 与 G_j 的条件协方差。利用马尔可夫性：给定 $S_j = s$ （第 j 步访问时的状态为 s ），未来的回报 G_j 与过去累积的 A_{ij} 条件独立。因此：

$$\begin{aligned} \text{Cov}(G_i, G_j \mid s, a) &= \mathbb{E}[(A_{ij} + \gamma^d G_j - q_\pi)(G_j - q_\pi) \mid s, a] \\ &= \mathbb{E}[A_{ij}(G_j - q_\pi) \mid s, a] + \gamma^d \text{Var}(G_j \mid s, a). \end{aligned} \quad (4)$$

其中 $q_\pi = q_\pi(s, a)$ 为真实动作价值。

第一项可进一步分解。记 $p_\pi^{(d)}(s' \mid s)$ 为从状态 s 出发经过 d 步转移后到达 s' 的概率（遵循策

略 π), a'_j 为第 j 步实际执行的动作。则:

$$\mathbb{E}[A_{ij}(G_j - q_\pi) | s, a] = \sum_{s'} p_\pi^{(d)}(s' | s) \sum_{a'} \pi(a' | s') \mathbb{E}[A_{ij} | s, a, S_j = s', A_j = a'] \cdot (q_\pi(s', a') - q_\pi(s, a)).$$

虽然该耦合项一般不为零, 但其大小受限于 $|q_\pi(\cdot, \cdot) - q_\pi(s, a)|$ 。在许多实际问题中, 若两次访问的中间状态在价值意义上与 s 差异有限, 该项的量级相对较小。

衰减上界的简化估计: 令 $\bar{A}_d = \max_{i,j} |\mathbb{E}[A_{ij} | s, a]|$ 为 d 步累积奖励期望的最大绝对值, 令 Δ_q 为动作价值函数的最大差异。则:

$$|\text{Cov}(G_i, G_j)| \leq (\bar{A}_d \cdot \Delta_q + \gamma^d \text{Var}(G_j)). \quad (5)$$

由于 $\bar{A}_d = O(\frac{1-\gamma^d}{1-\gamma})$ 是 d 的有界函数, 当 $\gamma^d \ll 1$ 时交叉项被 γ^d 主导。因此多次访问之间的协方差以 γ^d 为主衰减项, 即:

$$\text{Cov}(G_i, G_j | s, a) \approx \gamma^{|i-j|} \cdot \text{Var}(G | s, a). \quad (6)$$

3. 相关系数

令 $\rho_d = \text{Corr}(G_i, G_j)$ 为间隔 d 步的两次访问的相关系数。由式(6), 近似有:

$$\rho_d \approx \gamma^d.$$

例如, 当 $\gamma = 0.9$ 时:

间隔 d	1	2	5	10	20	50
ρ_d	0.90	0.81	0.59	0.35	0.12	0.005

可见相关性随间隔 d 呈指数衰减。间隔超过约 $\frac{-\ln 0.1}{\ln(1/\gamma)} \approx 22$ 步后 (对 $\gamma = 0.9$), 相关系数降至 0.1 以下, 样本可视为近似独立。

4. 有效样本量与方差比较

设共运行 n 个 episode, 每个 episode 平均包含 M 次对 (s, a) 的访问。首次访问策略使用 n 个样本; 每次访问策略使用 nM 个样本, 但样本间存在相关性。

对于每次访问策略, 估计量 $\hat{q}_{\text{ev}} = \frac{1}{nM} \sum_{i=1}^{nM} G_{(i)}$ 的方差为 (假设 M 保持不变以简化分析):

$$\text{Var}(\hat{q}_{\text{ev}}) = \frac{\sigma^2}{nM} \left(1 + \frac{2}{M} \sum_{1 \leq i < j \leq M} \rho_{|i-j|} \right) \approx \frac{\sigma^2}{nM} \left(1 + 2 \sum_{d=1}^{M-1} \frac{M-d}{M} \gamma^{\Delta d} \right), \quad (7)$$

其中 $\sigma^2 = \text{Var}(G | s, a)$ and $\Delta d \geq d$ 。定义**方差膨胀因子 (Variance Inflation Factor, VIF)** 为每次访问方差与独立同分布方差的比值:

$$\text{VIF}_M = 1 + 2 \sum_{d=1}^{M-1} \frac{M-d}{M} \gamma^{\Delta d}.$$

对于首次访问策略, $\text{Var}(\hat{q}_{\text{fv}}) = \sigma^2/n$ 。因此:

$$\frac{\text{Var}(\hat{q}_{\text{ev}})}{\text{Var}(\hat{q}_{\text{fv}})} = \frac{1}{M} \cdot \text{VIF}_M. \quad (8)$$

当 $M \rightarrow \infty$ 时, $\text{VIF}_M \rightarrow 1 + \frac{2\gamma}{1-\gamma} = \frac{1+\gamma}{1-\gamma}$ 。因此方差比趋近于:

$$\frac{\text{Var}(\hat{q}_{\text{ev}})}{\text{Var}(\hat{q}_{\text{fv}})} \rightarrow \frac{1+\gamma}{M(1-\gamma)}.$$

效率判别准则: 每次访问策略的方差低于首次访问策略当且仅当:

$$M > \frac{1+\gamma}{1-\gamma}.$$

即每个 episode 中对同一 (s, a) 的平均访问次数超过该阈值时, 每次访问策略更优。

典型取值:

γ	0.8	0.9	0.95	0.98	0.99
阈值 $\frac{1+\gamma}{1-\gamma}$	9	19	39	99	199

当 γ 较大（接近 1）时，阈值较高，首次访问策略在 episode 较短时更有优势；当 γ 较小时，每次访问策略很快就能凭借样本数量优势获得更低的方差。

5. 结论

上述分析给出以下定量结论：

- **相关性衰减**：同 episode 内两次 (s, a) 访问的回报相关系数近似为 γ^d ，以折扣因子为底数呈指数衰减。
- **有效样本折扣**：当每个 episode 对 (s, a) 访问 M 次时，有效独立样本数约为 $n_{\text{eff}} = \frac{nM(1-\gamma)}{1+\gamma}$ ，而非 nM 。
- **效率阈值**：仅当 $M > \frac{1+\gamma}{1-\gamma}$ 时，每次访问策略的估计方差才低于首次访问策略。这一结果解释了为什么在折扣因子接近 1 的长期规划任务中，首次访问策略往往更可取。

5.3.2 更高效地更新策略

MC 强化学习的另一个方面是何时更新策略。有两种策略：

第一种策略（MC Basic 采用）：在策略评估步骤中，先收集所有从同一状态-动作对出发的 episode，然后用这些 episode 的平均回报来近似动作价值。其缺点在于：智能体必须等待所有 episode 收集完成后才能更新估计。

第二种策略：用单个 episode 的回报来近似对应的动作价值。这样可以在收到每个 episode 时立即获得粗略估计，然后以逐 episode 的方式改进策略。该策略属于第 4 讲介绍的广义策略迭代的范畴——即使价值估计还不够准确，也可以更新策略。

5.3.3 算法描述

将 5.3.1 节和 5.3.2 节的技术结合起来增强 MC Basic 的效率，就得到了 **MC Exploring Starts** 算法。

定义

算法 5.2: MC Exploring Starts (MC Basic 的高效变体)

初始化：对所有 (s, a) ，初始化策略 $\pi_0(a | s)$ 和初始值 $q(s, a)$ ；

$\text{Returns}(s, a) = 0$ ， $\text{Num}(s, a) = 0$ 。

目标：搜索最优策略。

对每个 episode，执行：

- **Episode 生成**：选择一个起始状态-动作对 (s_0, a_0) ，并确保所有对都可能被选中（exploring starts 条件）。遵循当前策略，生成长度为 T 的 episode: $s_0, a_0, r_1, \dots, s_{T-1}, a_{T-1}, r_T$ 。
- **每个 episode 的初始化**： $g \leftarrow 0$ 。
- 对 episode 中的每一步， $t = T - 1, T - 2, \dots, 0$ ，执行：
 - $g \leftarrow \gamma g + r_{t+1}$
 - $\text{Returns}(s_t, a_t) \leftarrow \text{Returns}(s_t, a_t) + g$
 - $\text{Num}(s_t, a_t) \leftarrow \text{Num}(s_t, a_t) + 1$
 - **策略评估**： $q(s_t, a_t) \leftarrow \text{Returns}(s_t, a_t) / \text{Num}(s_t, a_t)$
 - **策略改进**： $\pi(a | s_t) = 1$ 若 $a = \arg \max_a q(s_t, a)$ ，否则 $\pi(a | s_t) = 0$ 。

MC Exploring Starts 采用了每次访问策略。在计算从每个状态-动作对出发的折扣回报时，过

程从终止状态开始逆向回到起始状态，这种技术可以提高算法效率，但也使算法更加复杂。正因如此，我们先介绍了不含这些技术的 MC Basic，以揭示 MC 强化学习的核心思想。

Exploration starts 条件要求从每个状态-动作对出发有足够多的 episode。只有每个状态-动作对被充分探索，才能准确估计其动作价值（大数定律），从而成功找到最优策略。否则，若某个动作未被充分探索，其动作价值可能被不准确地估计，即使它确实是最好的动作，也可能不被策略选中。MC Basic 和 MC Exploring Starts 都需要这一条件，但它在许多应用中（尤其涉及与环境的物理交互时）难以满足。

5.4 MC ϵ -Greedy: 无需 Exploring Starts 的学习

接下来扩展 MC Exploring Starts 算法，移除 exploring starts 条件。该条件本质上要求每个状态-动作对被足够多次访问，这也可以基于软策略（soft policies）来实现。

5.4.1 ϵ -greedy 策略

一个策略若在任意状态下有正概率选择任意动作，则称为**软策略（soft policy）**。考虑一个极端情形：仅有一个 episode。使用软策略时，一个足够长的 episode 可以多次访问每个状态-动作对，因此不需要从不同状态-动作对出发生成大量 episode，exploring starts 要求因而可以被移除。

一种常见的软策略是 **ϵ -greedy 策略**。 ϵ -greedy 策略是一种随机策略，它有更高的概率选择贪心动作，并以相同的非零概率选择其他任何动作。具体地，设 $\epsilon \in [0, 1]$ ，对应的 ϵ -greedy 策略形式为：

$$\pi(a | s) = \begin{cases} 1 - \frac{|\mathcal{A}(s)| - 1}{|\mathcal{A}(s)|} \epsilon, & \text{对于贪心动作,} \\ \frac{1}{|\mathcal{A}(s)|} \epsilon, & \text{对于其他 } |\mathcal{A}(s)| - 1 \text{ 个动作,} \end{cases} \quad (9)$$

其中 $|\mathcal{A}(s)|$ 表示状态 s 关联的动作数量。

当 $\epsilon = 0$ 时， ϵ -greedy 退化为贪心策略；当 $\epsilon = 1$ 时，选择任意动作的概率均为 $1/|\mathcal{A}(s)|$ 。

贪心动作的被选概率始终大于其他动作，因为对于任意 $\epsilon \in [0, 1]$ ：

$$1 - \frac{|\mathcal{A}(s)| - 1}{|\mathcal{A}(s)|} \epsilon = 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}(s)|} \geq \frac{\epsilon}{|\mathcal{A}(s)|}.$$

虽然 ϵ -greedy 策略是随机的，但按如下方式选择动作：首先生成 $[0, 1]$ 上的均匀随机数 x 。若 $x \leq \epsilon$ ，则在 $\mathcal{A}(s)$ 中随机均匀选择一个动作（可能再次选到贪心动作）；若 $x > \epsilon$ ，则选择贪心动作。这样，选择贪心动作的总概率为 $1 - \epsilon + \epsilon/|\mathcal{A}(s)|$ ，选择其他任何动作的概率为 $\epsilon/|\mathcal{A}(s)|$ 。

5.4.2 算法描述

要将 ϵ -greedy 策略集成到 MC 学习中，只需将策略改进步骤从贪心改为 ϵ -greedy。

具体地，MC Basic 或 MC Exploring Starts 中的策略改进步骤求解：

$$\pi_{k+1}(s) = \arg \max_{\pi \in \Pi} \sum_a \pi(a | s) q_{\pi_k}(s, a),$$

其中 Π 表示所有可能策略的集合。该问题的解为贪心策略。

现在，将策略改进步骤改为求解：

$$\pi_{k+1}(s) = \arg \max_{\pi \in \Pi_\epsilon} \sum_a \pi(a | s) q_{\pi_k}(s, a),$$

其中 Π_ϵ 表示给定 ϵ 值下所有 ϵ -greedy 策略的集合。这样将策略强制为 ϵ -greedy 的。该问题的解为式(9)，其中 $a_k^* = \arg \max_a q_{\pi_k}(s, a)$ 。

定义

算法 5.3: MC ϵ -Greedy (MC Exploring Starts 的变体)

初始化: 对所有 (s, a) , 初始化策略 $\pi_0(a | s)$ 和初始值 $q(s, a)$;

Returns(s, a) = 0, Num(s, a) = 0; $\epsilon \in (0, 1]$ 。

目标: 搜索最优策略。

对每个 episode, 执行:

- **Episode 生成:** 选择一个起始状态-动作对 (s_0, a_0) (不需要 exploring starts 条件)。遵循当前策略, 生成长度为 T 的 episode: $s_0, a_0, r_1, \dots, s_{T-1}, a_{T-1}, r_T$ 。
- **每个 episode 的初始化:** $g \leftarrow 0$ 。
- 对 episode 中的每一步, $t = T - 1, T - 2, \dots, 0$, 执行:
 - $g \leftarrow \gamma g + r_{t+1}$
 - Returns(s_t, a_t) $\leftarrow$ Returns(s_t, a_t) + g
 - Num(s_t, a_t) $\leftarrow$ Num(s_t, a_t) + 1
 - **策略评估:** $q(s_t, a_t) \leftarrow$ Returns(s_t, a_t)/Num(s_t, a_t)
 - **策略改进:** 令 $a^* = \arg \max_a q(s_t, a)$, 则

$$\pi(a | s_t) = \begin{cases} 1 - \frac{|\mathcal{A}(s_t)| - 1}{|\mathcal{A}(s_t)|} \epsilon, & a = a^*, \\ \frac{1}{|\mathcal{A}(s_t)|} \epsilon, & a \neq a^*. \end{cases}$$

若在策略改进步骤中用 ϵ -greedy 策略替换贪心策略, 还能保证获得最优策略吗? 答案是“是”也是“否”。“是”是指: 当有足够样本时, 该算法可以收敛到 Π_ϵ 中的最优 ϵ -greedy 策略。“否”是指: 该策略仅在 Π_ϵ 中是最优的, 但在 Π 中可能不是最优的。然而, 如果 ϵ 足够小, Π_ϵ 中的最优策略会接近 Π 中的最优策略。

5.4.3 示例分析

考虑图 5.5 所示的网格世界。目标是为每个状态找到最优策略。MC ϵ -Greedy 算法的每次迭代生成一个一百万步的 episode (这里故意考虑仅用一个 episode 的极端情形)。奖励设置为 $r_{\text{boundary}} = r_{\text{forbidden}} = -1$, $r_{\text{target}} = 1$, $\gamma = 0.9$ 。

初始策略是均匀策略 (每个动作的概率均为 0.2)。 $\epsilon = 0.5$ 的最优 ϵ -greedy 策略经过两次迭代即可得到。虽然每次迭代仅使用一个 episode, 但由于所有状态-动作对都能被访问到, 其价值可以被准确估计, 策略逐步改进。

5.5 ϵ -Greedy 策略的探索与利用

探索 (exploration) 与 **利用 (exploitation)** 是强化学习中的基本权衡。探索意味着策略尽可能多地尝试不同动作, 从而充分访问和评估所有动作。利用意味着改进后的策略应选择动作价值最大的贪心动作。然而, 由于当前时刻获得的动作价值可能因探索不充分而不够准确, 我们需要在利用的同时保持探索, 以避免错过最优动作。

ϵ -greedy 策略提供了一种平衡探索与利用的方式。一方面, 它以更高概率选择贪心动作以利用当前估计的价值; 另一方面, 它也有机会选择其他动作以持续探索。

利用与最优性相关——最优策略应当是贪心的。 ϵ -greedy 策略的基本思想是以牺牲最优性/利用为代价来增强探索。要增强利用和最优性，需减小 ϵ ；要增强探索，需增大 ϵ 。

ϵ -Greedy 策略的最优性

下面通过 5×5 网格世界示例展示：随着 ϵ 增大， ϵ -greedy 策略的最优性变差。奖励设置为 $r_{\text{boundary}} = -1$, $r_{\text{forbidden}} = -10$, $r_{\text{target}} = 1$, $\gamma = 0.9$ 。

- $\epsilon = 0$ 时为贪心最优策略，状态价值最高。目标状态的价值也达到最大。
- 随着 ϵ 增大，状态价值逐渐降低， ϵ -greedy 策略的最优性变差。值得注意的是，当 ϵ 大到 0.5 时，目标状态的价值反而变得最小——这是因为 ϵ 较大时，从目标区域出发的智能体有更高概率进入周边禁区并收到负奖励。
- 如图 5.7 所示，当 $\epsilon = 0.1$ 时，最优 ϵ -greedy 策略与最优贪心策略一致。但当 ϵ 增大到 0.2 时，获得的 ϵ -greedy 策略与最优贪心策略不再一致。因此，若希望获得与最优贪心策略一致的 ϵ -greedy 策略， ϵ 的值应足够小。

为什么 ϵ 较大时 ϵ -greedy 策略与最优贪心策略不一致？以目标状态为例：贪心情形下，目标状态的最优策略是保持不动以获得正奖励；但当 ϵ 较大时，有 high 概率进入禁区并获得负奖励，因此此时目标状态的最优策略是逃离而非停留。

ϵ -Greedy 策略的探索能力

下面通过示例说明： ϵ 越大， ϵ -greedy 策略的探索能力越强。

当 $\epsilon = 1$ 时（均匀策略），探索能力最强——在任意状态有 0.2 的概率选择任意动作。从 (s_1, a_1) 出发生成的一个 episode，当 episode 足够长时（如 100 万步），可以多次访问所有状态-动作对，且访问次数几乎均匀分布。

当 $\epsilon = 0.5$ 时，探索能力弱于 $\epsilon = 1$ 的情形。虽然 episode 足够长时每个动作仍能被访问到，但访问次数的分布极不均匀——某些动作被访问超过 25 万次，而大多数动作仅被访问几百甚至几十次。

以上示例表明： ϵ 减小时， ϵ -greedy 策略的探索能力下降。一种有用的技术是：初始时设 ϵ 较大以增强探索，然后逐步减小 ϵ 以保证最终策略的最优性。

备注

ϵ -greedy 策略不仅用于 MC 强化学习，也用于其他强化学习算法（如第 7 讲将介绍的时序差分学习）。探索与利用的权衡是贯穿整个强化学习领域的核心主题。

5.6 本章小结

本章介绍了本书中第一个无模型强化学习算法系列。首先通过均值估计问题引入了 MC 估计的思想；然后依次介绍了三种 MC 算法：

- **MC Basic:** 最简单的 MC 强化学习算法。该算法通过将策略迭代中基于模型的策略评估步骤替换为无模型的 MC 估计组件而得到。给定足够样本，该算法可收敛到最优策略和最优状态价值。
- **MC Exploring Starts:** MC Basic 的变体。通过采用首次访问或每次访问策略更高效地利用样本，并从 MC Basic 得到该算法。
- **MC ϵ -Greedy:** MC Exploring Starts 的变体。在策略改进步骤中搜索最优 ϵ -greedy 策略而非贪心策略，从而增强了策略的探索能力，移除了 exploring starts 条件。

最后，通过分析 ϵ -greedy 策略的性质介绍了探索与利用的权衡：增大 ϵ 可增强探索能力但降低利用/最优性，减小 ϵ 可更好利用贪心动作但探索能力受损。

5.7 常见问题解答

- **Q: 什么是蒙特卡洛估计?**
A: 蒙特卡洛估计是指使用随机样本来解决近似问题的一大类技术。
- **Q: 什么是均值估计问题?**
A: 均值估计问题是指基于随机样本来计算随机变量期望值的问题。
- **Q: 如何求解均值估计问题?**
A: 有两种方法——基于模型和无模型。若随机变量的概率分布已知, 可根据定义直接计算期望; 若概率分布未知, 可使用 MC 估计来近似期望, 当样本数量足够大时该近似非常准确。
- **Q: 为什么均值估计问题对强化学习很重要?**
A: 状态价值和动作价值都定义为回报的期望, 因此估计状态价值或动作价值本质上就是均值估计问题。
- **Q: 无模型 MC 强化学习的核心思想是什么?**
A: 核心思想是将策略迭代算法改造为无模型算法。策略迭代基于系统模型计算价值, 而 MC 强化学习将策略迭代中基于模型的策略评估步骤替换为无模型的 MC 策略评估步骤。
- **Q: 什么是初始访问、首次访问和每次访问策略?**
A: 它们是利用 episode 中样本的不同策略。一个 episode 可能访问许多状态-动作对。初始访问策略用整个 episode 仅估计起始状态-动作对的动作价值。每次访问和首次访问策略能更好地利用样本——若每次访问状态-动作对时都用其后轨迹估计其动作价值, 则为每次访问策略; 若只使用首次访问, 则为首次访问策略。
- **Q: 什么是 exploring starts? 为什么它很重要?**
A: Exploring starts 要求从每个状态-动作对出发生成无限多 (或足够多) 的 episode。理论上该条件是找到最优策略所必需的——只有每个动作价值被充分探索, 才能准确评估所有动作并正确选择最优动作。
- **Q: 用什么思想避免 exploring starts?**
A: 基本思想是让策略变“软”。软策略是随机的, 使一个 episode 能访问许多状态-动作对, 从而不需要从每个状态-动作对出发的大量 episode。
- **Q: ϵ -greedy 策略可以是最优的吗?**
A: 答案既是“是”也是“否”。“是”是指: 若有足够样本, MC ϵ -Greedy 算法可收敛到最优 ϵ -greedy 策略。“否”是指: 收敛的策略仅在所有 ϵ -greedy 策略中是最优的 (对相同的 ϵ 值)。
- **Q: 能否用一个 episode 访问所有状态-动作对?**
A: 可以。若策略是软的 (如 ϵ -greedy) 且 episode 足够长, 一个 episode 即可访问所有状态-动作对。
- **Q: MC Basic、MC Exploring Starts 和 MC ϵ -Greedy 之间是什么关系?**
A: MC Basic 是最简单的 MC 强化学习算法, 它揭示了无模型 MC 强化学习的基本思想。MC Exploring Starts 是 MC Basic 的变体, 调整了样本使用策略。MC ϵ -Greedy 又是 MC Exploring Starts 的变体, 移除了 exploring starts 要求。因此, 虽然基本思想简单, 但当追求更好性能时, 复杂性随之增加。将核心思想与可能分散初学者注意力的复杂性分开, 对于理解至关重要。